约束即所有所需：基于优化的3D关卡生成与大型语言模型

徐凯捷 麦吉尔大学 蒙特利尔，魁北克，加拿大
kaijie.xu2@mail.mcgill.ca

克拉克·韦布鲁格 麦吉尔大学 蒙特利尔，魁北克，加拿大
clump@cs.mcgill.ca

摘要

程序化内容生成（PCG）长期以来一直支持高效且多样化的游戏关卡创建。然而，将高级设计意图和游戏机制集成到复杂的3D环境中仍然具有挑战性。本文介绍了一个综合框架，将叙事级描述转换为可玩的3D游戏关卡。首先，大型语言模型（LLMs）将游戏环境的自然语言描述解析为结构化的游戏关卡描述语言（GLDL），捕获关键的空间约束。接下来，我们将关卡生成建模为设施布局优化问题，确保设施布局和配置符合指定的设计标准。通过全面的实验，包括自动化的约束评估和基于代理的模拟，我们的方法确保从文本描述中提取的约束的可行性和稳定性。我们确认生成的游戏关卡保持交互性、合理性和可控性，符合原始规格。

CCS 概念

• **应用计算 →计算机游戏**。

关键词

程序化内容生成，设施布局问题，关卡生成，大语言模型，游戏设计

ACM 参考格式：
徐凯捷和克拉克·韦布鲁格。2025。约束即所有：基于优化的3D关卡生成与大语言模型。收录于《国际数字游戏基础会议 *(FDG '25)，2025*年*4*月*15*日至*18*日，格拉茨，奥地利》。ACM，纽约，纽约，美国，13页。
https://doi.org/10.1145/3723498.3723840

1 引言

程序化内容生成（PCG）在学术界和工业界都是一个持续研究和不断发展的主题，它能够自动创建多样、可重玩且具有丰富上下文的游戏环境，特别是在roguelike和开放世界游戏中。然而，复杂的游戏世界需要能够整合高级设计意图的PCG系统，以确保叙事连贯性和游戏机制的有效运用。在叙事驱动的关卡设计中，物体的空间排列和角色的分布必须与整体故事和游戏目标保持一致。通过融入这些元素，PCG工具使设计师能够在保持创意控制的同时，实现功能强大且视觉协调的环境。

本作品根据知识共享署名国际 4.0 许可协议授权使用。

FDG '25, 格拉茨，奥地利©2025 版权由所有者/作者持有。ACM ISBN 979-8-4007-1856-4/25/04
https://doi.org/10.1145/3723498.3723840

和角色的分布必须与整体故事和游戏目标保持一致。通过融入这些元素，PCG工具使设计师能够在保持创意控制的同时，实现功能强大且视觉协调的环境。

近年来，在大语言模型（LLMs）与计算机视觉（CV）技术相结合方面的最新进展已证实，在生成视觉上吸引人的场景方面取得了实质性进展 [1, 5, 6,10, 17–19, 27–29]。然而，将这些方法应用于游戏关卡生成时，它们并不总是充分考虑到游戏逻辑、机制和交互元素等关键方面。因此，尽管它们在纯场景生成方面很有效，但它们可能产生视觉上引人入胜但缺乏功能一致性的环境。因此，我们寻求一个更通用的框架，该框架支持自定义资源集成，同时优先考虑结构一致性和功能可玩性。

在本工作中，我们提出了一种新颖的框架，该框架利用大语言模型的能力，以弥合叙事描述与结构化关卡设计之间的差距。这种方法引入了一种专门的游戏关卡描述语言（GLDL），该语言对从自然语言输入中推导出的基本空间约束、关系和游戏机制进行编码。通过将关卡生成表述为一个设施布局优化问题——这是一个经典的挑战，专注于在给定空间内排列设施以优化空间效率和满足设计约束——该框架将叙事驱动的规范转化为精确且可行的3D布局。这种方法论确保生成的环境不仅视觉上协调一致，而且符合所需的游戏逻辑和交互动态。

我们工作的主要贡献包括：

- 我们描述了一个从文本输入生成非平凡游戏关卡的程序化内容生成（PCG）流程，该流程结合了基本的游戏机制，如锁/钥匙或其他分布式谜题、战斗和空间排列。
- 我们的流程依赖于一种新颖的游戏关卡描述语言（GLDL），这是一种受Scenic [7]启发的专门语言，旨在将空间约束、关系和游戏机制作为从自然语言描述中推导出的内容进行编码。这促进了将叙事意图转化为结构化关卡规范的过程。
- 将GLDL翻译为实际关卡结构被建模为一个基本的设施布局优化问题。我们探索各种优化方法，能够自动放置符合叙事和游戏玩法约束的游戏设施。
- 我们的工作通过Unity3D模拟得到验证，采用合成智能体方法确保生成关卡的可玩性和连贯性。

2 相关工作

关卡生成是游戏研究领域的一个大型且历史悠久的研究方向。以下，我们总结了与关卡生成、设施布局问题以及大语言模型在关卡生成中的应用相关的前人工作。

2.1 过程式内容生成(PCG)与关卡生成

过程式内容生成 (PCG) 在游戏设计领域已被广泛探索，强调关卡、地图和场景等游戏元素的自動创建。传统的 PCG 方法包括随机生成、基于规则的系统和基于搜索的方法，每种方法都有其独特的优势和局限性 [20, 24]。近年来，研究趋势显著转向整合机器学习和强化学习技术，以增强生成内容的适应性、复杂性和创造性 [12, 13, 22]。

虽然二维、基于网格的关卡生成已取得长足进展，但过程式内容生成在三维环境中的探索仍相对有限。一些研究采用强化学习技术构建三维环境 [11]。此外，计算机视觉方法在其原生领域已展现出显著效能，并被应用于三维场景的生成 [5, 6, 17–19, 28]。我们的研究旨在通过开发一个利用大语言模型和优化算法的框架，来填补这一空白，以创建可玩、多样且连贯的3D游戏关卡。

2.2 设施布局问题

设施布局问题（FLP）是运筹学中一个成熟的优化挑战，聚焦于在给定空间内排列设施以优化特定标准，例如最小化移动成本或最大化空间利用率 [2, 21]。传统上应用于制造业和服务业，FLP通过确定各种设施或部门的最有效位置来提升运营效率和空间组织。FLP于2006年引入游戏领域，主要聚焦于生成建筑内部和建筑房间布局 [8, 14, 15]。

近年来，FLP已将其应用扩展到传统产业以外的领域，包括交通场景生成和2D游戏世界生成等场景生成任务 [4, 25, 26]。通过将FLP方法应用于这些新领域，研究人员可以在复杂环境中实现元素的系统化和优化布局。我们的研究利用FLP来解决3D游戏关卡生成中固有的挑战。通过将关卡设计表述为FLP，我们可以将真实约束模拟为抽象的数值约束，并采用先进的优化算法来确定符合游戏机制和视觉设计原则的设施最优布局。

2.3 大语言模型在程序化内容生成中的应用

大型语言模型（LLMs）通过数十亿个参数进行预训练，以执行广泛的任务，包括自然语言理解和生成 [30]。最近的进展，例如 GPTs [16] 和 Claude [3], , 已经展示了卓越文本生成能力，突显了人工通用智能的潜力。在程序化内容生成（PCG）领域，大语言模型通过解释基于文本的提示来生成规则和关卡 [9]。大语言模型通过自然语言提示提供直观的界面，与玩家轨迹等传统方法相比，使人更容易参与和提供反馈。最新研究 [1, 10, 29]进一步扩展了大语言模型在生成复杂3D场景和沉浸式室内环境方面的能力。尽管最近有努力在固定游戏规则和数据集内微调大语言模型进行关卡生成 [23], , 但这些方法通常需要额外的训练时间，并且容易受到大语言模型固有的随机性和幻觉的影响。在我们的研究中，我们利用大语言模型将游戏关卡描述文本转换为专门的游戏关卡描述语言（GLDL），从而最大限度地减少直接大语言模型输出的不稳定性，并提高生成游戏关卡的可靠性。

3 方法论

我们的程序化内容生成框架旨在通过集成工作流程，高效地将自然语言描述转换为可玩的游戏关卡。如图1所示，该过程始于输入高级、人类编写的叙事描述，然后由大语言模型处理以提取关键词和约束。这些元素被转换为结构化的游戏关卡描述语言（GLDL），指导布局优化算法生成房间内设施放置的详细数据。这些数据随后用于模拟最终游戏关卡。为确保生成关卡的质量和稳定性，该框架包括“大语言模型稳定性测试”，评估大语言模型生成内容的连贯性和一致性，以及“智能体模拟测试”，通过模拟智能体在生成环境中的行为来评估可玩性和用户体验。这种系统化方法确保生成的关卡不仅上下文准确，而且可玩。

下面我们先介绍GLDL，然后描述使用LLM作为约束生成器的过程，以及我们探索的不同最终布局生成方法。LLM稳定性和代理模拟测试是实验验证的一部分，因此将在下一节中描述。

3.1 游戏关卡描述语言（GLDL）

GLDL，即游戏关卡描述语言，是一种描述性语言，部分灵感来源于更通用的Scenic语言 [7]，但更具体地聚焦于系统地表示三维游戏关卡。通过提供定义设施、空间变量和约束的结构化框架，GLDL促进了连贯、具有上下文意义且可玩环境的自动化创建。设施包括游戏中可能出现的所有对象，包括基本建筑元素、交互功能和非玩家角色（NPC）。本节介绍了GLDL的核心组件，阐明了基本句子类型，对约束进行了分类，并突出了采用GLDL的实际优势。更复杂的语言定义在附录A中给出。

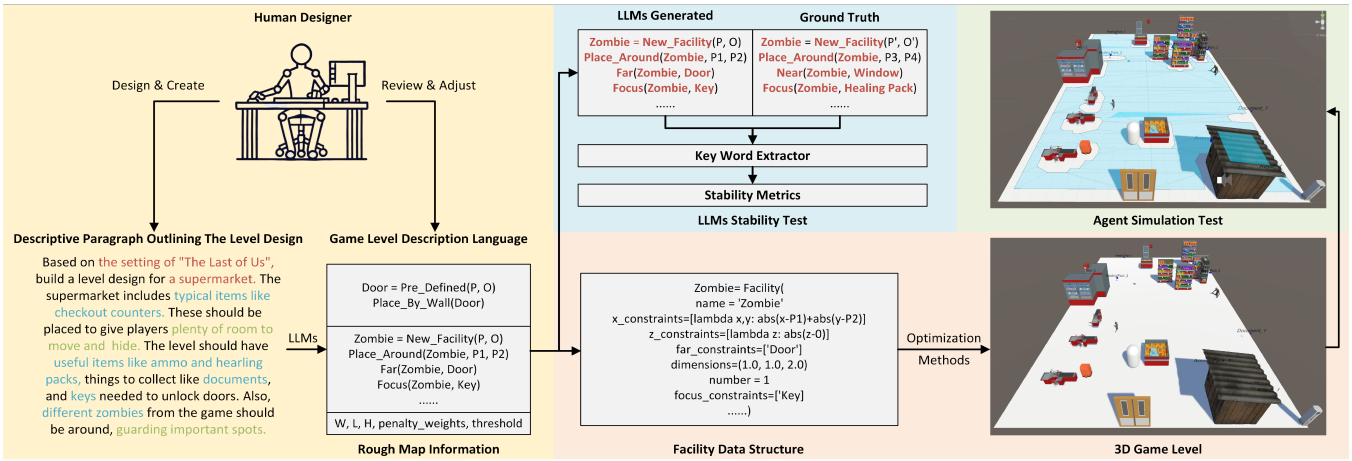


图1：我们程序化内容生成框架的整体工作流程：自然语言段落描述目标3D环境，由LLM处理成GLDL约束（黄色，左侧）；这些约束通过LLM稳定性测试（蓝色，上中部分）进行验证，其中红色文本突出显示关键组件（设施名称、约束类型、变量类型），然后输入优化算法以生成设施布局（米色，右下部分）；最后，在Unity中进行智能体模拟测试（绿色，右上部分），验证生成的3D关卡满足预期的可导航性、复杂性和整体设计要求。

3.1.1 词汇和核心组件。 GLDL 基本上基于设施的概念，提供了一种描述游戏中的对象、NPC 和其他实体相关约束和需求的方法。该语言由两种主要类型的句子组成：定义句子和约束句子。定义句子用于声明和参数化设施，有效地建立游戏关卡的基本组件。随后，约束句子用于对这些定义的设施施加特定的约束和要求。典型的工作流程首先使用适当的定义句子定义每个对象，然后应用与其个体特征相关的基本约束，例如位置要求或房间内的空间从属关系。当引入与现有设施交互的新设施时，会添加关系约束句子来定义它们的相互依赖和空间关系。这种结构化方法确保每个设施不仅被准确定义，而且被连贯地集成到整体游戏环境中。

定义和约束句子。 GLDL规范由两种主要句子类型组成：

- **定义句子：** 这些句子声明和参数化设施（对象或参考点），格式如下：

$$F = \text{定义}(\text{变量})$$

在此上下文中， F表示一个设施，而<样式 id='2'>定义</样式>作为特定类型定义句子的占位符。以下小节描述了GLDL支持的三个定义句子变体：

– 预定义设施: 一个具有预定位置和朝向的设施，作为稳定的参考点。此类设施在优化过程中不被视为变量。

– 新设施: 一个其位置和朝向取决于指定约束的设施，能够实现自适应和动态布局。新设施在优化中被视为变量。– 点: 特殊的设施，作为固定的参考锚点，指导其他设施相对位置和朝向。

- <样式 id='1'>约束句子：</样式>这些句子对设施施加空间或关系规则，使用以下格式：

约束<样式 *id='1'*>(</样式>*F*, 变量<样式 *id='4'*>)</样式>

在此表达式中， F 表示一个设施，并列出的变量定义了它如何与位置、其他设施或环境参数相关。约束被分为两种主要类型，以简化规范并促进对设施布局的推理：

<样式 id='1'>– 基本约束</样式>：定义相对于环境或指定参考点的基本放置规则。例如包括<样式 id='3'>围绕位置放置</样式>、<样式 id='5'>在范围内放置</样式>和<样式 id='7'>靠墙放置</样式>。<样式 id='9'>– 关系约束</样式>：控制设施之间的空间和方向关系，涉及邻近、可视和朝向等方面。例如包括<样式 id='11'>邻近/远离</样式>、<样式 id='13'>可视</样式>、<样式 id='15'>聚焦</样式>和<样式 id='17'>对齐/朝向</样式>。不同类型约束句子的更多细节和规则在附录A中提供。

变量。 变量代表定义设施和指导其配置的基本属性和参数（例如，位置、朝向、标签、数量）。它们出现在定义和约束句中，确保规范能够随着环境或设计需求随时间变化而灵活和可扩展。

3.1.2 GLDL 的优势。 GLDL 为设计师和自动化流程提供了多种优势:

领域特定简单性：GLDL的构造是为典型游戏设计场景而设计的，使其比旨在用于复杂感知任务的通用规范语言更直接和易于理解。

可访问性对设计师和大语言模型： GLDL的简单组件和语法可以被没有广泛技术专长的设计师轻松理解。此外，大语言模型可以从简洁的提示中解释和生成GLDL代码，从而促进快速初始开发，实现迭代改进，并支持协作设计流程。虽然GLDL设计了一种无需专家知识即可理解其底层本体论的易于访问的语法，但我们的当前评估假设用户提供自由文本输入，而无需完全理解详细的定义和约束结构。未来的工作将涉及对不同设计师群体进行用户研究，以更好地量化有效使用所需熟悉程度。

3.2 基于大语言模型的约束生成

借助大语言模型（LLM），我们的方法将游戏关卡的自然语言描述转换为GLDL。该过程涉及几个关键步骤:

3.2.1 提示工程。 在为基于大语言模型的约束生成设计提示时，我们构建提示以清晰地概述目标、组件以及生成GLDL合规语法所需的特定指令。这涉及向LLM提供游戏环境的简洁描述，定义设施类型、其属性以及它们之间的关系。通过明确指定GLDL的目的、关键组件以及定义和约束句子的语法，我们确保大部分生成输出符合所需格式并准确反映预期的关卡设计。例如，在创建游戏关卡时，提示定义了设施类型，如预定义设施（PredefinedFacility）和新建设施（NewFacility），以及它们的属性，如位置、朝向和尺寸。此外，提示还包括约束函数，如PlaceByWall和CanSee，以指导这些设施的放置和关系。重要的是，初始的叙事描述——由人类设计师（或由LLM生成）——与指令模板一起提供，该模板指导LLM将提示转换为结构化的GLDL代码。

要具体说明此提示结构的示例，请参见附录B。这些示例详细介绍了我们如何指导大语言模型创建后末日超市关卡，列举设施定义和约束，突出显示不使用引号的朝向用法，并引用环境变量。这确保生成的GLDL代码满足句法、空间和上下文要求，且只需最少的手动干预。

3.2.2 验证与精炼。 为了确保生成约束的准确性和可行性，我们实现了一个验证与精炼过程，其中每个生成的GLDL脚本都会被自动解析并对照参考语法和预定义的特定领域规则进行评估，以检测任何句法或语义上的偏差。大多数生成的句子在语义上是合理的并且无需调整；然而，这仅证实了它们在关卡构建中的有效性，而非它们满足我们所有设计预期或构建了一个完全可玩的关卡。任何差异或错误都通过额外的提示或手动调整进行迭代优化，以保持GLDL定义的完整性，而匹配率指标则进一步评估脚本满足我们定性标准的情况。在我们的实验中，由于句法错误而需要调整的句子比例不到0.01%，这是可以忽略不计的。

3.2.3 集成阶段。 验证后的GLDL输出随后集成到我们的优化流程中。这种从自然语言到结构化数据的平滑过渡，使得能够自动生成既多样化又符合上下文的游戏关卡。

3.3 优化算法

根据GLDL描述的连贯且可玩的3D游戏关卡生成，可以将其表述为一个带约束的优化问题。每个候选解对应于所有非预定义设施的位置和朝向参数的特定分配，而预定义设施则保持其固定配置。优化算法输出详细数据，指定每个设施的位置和朝向，这些数据随后被导入游戏引擎进行模拟。目标是找到一个设施配置，使其最小化一个综合指标，该指标反映了约束满足度、空间连贯性和环境质量。

3.3.1 问题公式化。 我们优化的核心是一个包含多个标准的目标函数:

- **空间约束满足：**对违反GLDL定义的约束施加惩罚，例如将设施放置在环境边界之外、物体重叠、错位、朝向错误，以及偏离期望的邻近或可见性条件。这些惩罚项被组合为空间惩罚 P，确保生成的布局紧密遵循指定的设计规则。

- **聚类和稀疏度指标：**除了原始约束满足外，我们还结合反映环境结构质量的指标。例如，聚类程度指标 C 可能会惩罚过度密集的放置，而稀疏度指标 S 则鼓励设施分布均衡。通过将这些指标与惩罚项加权，我们引导优化过程生成不仅可行，而且美观和功能上吸引人的布局。

我们的目标是找到设施的最佳位置和朝向，以最小化总指标。最终目标函数将这些项聚合为加权总和:

$$\text{TotalMetric} = w_p \cdot P + w_c \cdot C + w_s \cdot S$$

其中 w_p、 w_c 和 w_s 分别是惩罚项、聚类程度和稀疏度指标的权重； P 表示总惩罚， C 表示聚类程度， S 表示稀疏度指标:

- P **(总空间惩罚)**： P 是违反 GLDL 约束所导致的所有惩罚的累积和。这些惩罚（详见附录 D）包括但不限于将设施放置在指定边界之外、多个设施重叠、设施朝向错位以及未能满足邻近性或可见性要求。形式上，它可以表示为：

$$P = \sum_{\text{constraints}} PenaltyTerm$$

每个惩罚项量化了特定约束的违反程度，并适当加权以反映其在整体布局质量中的重要性。P 通过惩罚任何违反来确保遵循GLDL约束，从而保持布局的结构和功能完整性。有关惩罚项的更多详细信息，请参阅附录D。

- C **(聚类度)**： C 量化了环境中设施之间的聚类程度。对于每对放置的设施(f_i, f_j)，它计算它们之间欧几里得距离的倒数，并将这些值相加以获得聚类度：

$$C = \sum_{i<j} \frac{1}{\text{dist}(f_i, f_j) + \varepsilon}$$

其中$\text{dist}(f_i, f_j)$是设施f_i和 f_j之间的欧几里得距离， ε是一个小的常数以防止除以零。较高的 C 表示更紧密的聚类，这取决于设计目标。 C 会阻止设施过度聚类，促进在环境中更均匀和平衡的分布。

- S **(稀疏度指标)**: S 通过评估设施分布来衡量环境中空旷空间的程度。环境在离散网格点上进行采样，对于每个网格点 gp，计算其到任意设施的最小距离。稀疏度指标定义为这些最小距离的最大值：

$$S = \max_{\text{grid point } gp} \left(\min_{\text{facilities } f} \text{dist}(gp, f) \right)$$

这里，$\text{dist}(gp, f)$ 表示网格点 gp 到设施 f 的欧几里得距离。较低的 S 值表明设施分布更均匀，大型空旷区域更少。我们选择稀疏度指标而非众所周知的空间熵，因为它在大型3D优化中的计算效率更高，并且结果的最大空隙距离直观易懂。 S 通过鼓励设施均匀占据空间来防止大型空旷区域的存在，确保空间平衡并增强可导航性。

通过调整权重 w_p、 w_c 和 w_s，设计师和自动化流程可以优先考虑关卡设计的不同方面。优化过程旨在最小化总指标，从而实现满足约束、避免拥挤，并在环境中保持设施均匀分布的布局。值得注意的是，总指标中的聚类惩罚针对我们的生存恐怖设计背景进行了调整，其中密集的物品聚集会增强紧张感。然而，模块化框架允许设计师调整这些权重——例如，减少RPG 资源中心的聚类惩罚——甚至引入新的指标，例如MMO 的掉落物分配公平性或故事驱动游戏的叙事物品邻近度，以更好地匹配特定类型的优先级。

3.3.2 优化方法。 我们采用三种优化算法，并在测试阶段选择最有效的一种来设计设施布局，以最小化总指标并遵守空间约束：

- **模拟退火：** 模拟退火 (SA) 通过模拟物理退火过程来优化设施布局。从初始布局开始，SA 通过迭代修改设施位置和朝向来最小化总指标。在每一步，它评估新配置，并可能以概率接受可能增加指标的变化，由温度参数控制。这种方法允许 SA 逃离局部最优，逐步优化布局，以平衡约束满足 (P)、聚类避免 (C) 和空间平衡 (S)。

- **遗传算法：** 遗传算法 (GA) 通过进化框架来处理布局优化。每个候选布局被编码为一个染色体，其中基因代表设施参数。GA 在多代中运行，应用选择、交叉和变异来进化能够最小化总指标的布局。通过基于惩罚评估适应度，GA 有效地探索多样化的解决方案，并收敛到满足约束并保持空间一致性的高质量布局。

- **贪婪方法：** 贪婪方法通过在每次放置步骤中做出局部最优选择来逐步构建设施布局。在定位设施时，算法选择能立即降低总指标的位置和朝向，专注于根据当前部分布局最小化惩罚。虽然不能保证全局最优，但贪婪方法具有计算效率，适合快速布局生成或作为后续由模拟退火或遗传算法进一步优化的初始解和基线模型。

4 实验与分析

本节展示了一系列实验，旨在验证LLM生成的GLDL数据，评估我们优化算法的有效性，并通过基于Unity的模拟验证生成关卡的可玩性。实验分为三个主要阶段：

(1) LLM稳定性验证测试 (2) 优化测试 (3) Unity模拟测试

4.1 LLM稳定性验证测试

此实验评估了由大语言模型生成的GLDL脚本的一致性和可靠性。我们聚焦于基于“最后生还者”和“左4”背景设定的五个不同场景描述。

为了验证我们的方法在不同大语言模型能力下的鲁棒性和一致性，我们进行了两个额外的实验：使用o1-mini模型生成1,000个超市场景，以及使用GPT-4o模型生成另一个1,000个场景。

4.1.1 大语言模型提示和场景。我们向大语言模型提供了一份结构化的提示，概述了GLDL语法、规则和示例句子，具体见附录B。然后给大语言模型提供了五个不同的场景描述——一个基线场景（超市）和四个由大语言模型生成的场景（银行大厅、酒店大厅、废弃教堂、音乐节），并指示每个场景生成1,000个脚本。基线场景描述请参考附录C。

4.1.2 指标和相似度评估。为了评估生成的脚本，我们使用了一个定制的Python工具，该工具：

(1) 将每个句子分类到三个类别之一：定义（设施声明）、基本约束（例如，PlaceBy-Wall、PlaceInRange、PlaceAroundPosition）和关系约束（例如，邻近、可视、聚焦、对齐）。

(2) 解析并规范化句子以提取关键组件（设施名称、约束类型）。

(3) 通过将结构和关键元素与基准场景的基线真实脚本进行比较来衡量句子级别的相似度。基准真实脚本（超市场景）是通过迭代设计者调整人工策划的，以确保叙事和游戏玩法对齐，而其他场景的真实规范则通过基于结构多样性和约束完整性指标的自动过滤从大语言模型生成候选中程序化选择，随后通过代理模拟进行人工审查和可行性测试，以验证可玩性和逻辑一致性。

(4) 计算每个类别的出现频率和匹配率。匹配率通过确保每个句子使用正确的约束类型、有效的设施引用和参数值（在预期范围的20%以内），量化了生成句子在结构和语义上与基线GLDL对齐的百分比（匹配率 = 匹配句子 / 总句子 × 100%）。

该方法确保了对LLM在多大程度上准确复制GLDL规范的结构和语义进行定量评估。需要注意的是，较高的匹配率主要反映了LLM一致地再现预期GLDL结构的能力，这对于自动解析和下游优化至关重要。然而，由于约束有时可能冲突，匹配率不被视为唯一的品质指标，而是与定性评估和模拟测试结合使用，以评估整体一致性和可玩性水平。

4.1.3 结果与分析。 表1总结了所有五个场景中每个类别的平均匹配率。值得注意的是，超市（基线）场景显示出最高的整体相似度，

表1：每个场景的LLM稳定性验证结果

场景	定义 (%)	基本 (%)	相对 (%)
超市（基线）	91.77	51.03	54.30
银行大厅	73.69	52.63	18.09
酒店大厅	93.40	55.02	16.87
废弃教堂	82.89	64.83	17.71
音乐节	69.14	63.38	8.09
基线（o1-mini）	98.75	56.00	80.71
基线（4o）	94.58	53.33	71.43

while 完全由大语言模型生成的场景在基本约束和关系约束方面表现出更多变异性。为确保我们实验的鲁棒性和一致性，我们采用了不同版本的大语言模型进行了额外的验证测试。初始数据集包含 5,000 个由 GPT-4o-mini 生成的脚本，该模型以其处理复杂指令的局限性而闻名。补充实验旨在证明，即使使用能力较弱的模型，我们的框架也能保持合理的准确性，并且随着更先进的模型的使用，性能预计会得到提升，从而确保我们优化框架的一致性和鲁棒性。

我们进一步从所有5,000个脚本中汇总了句子频率，如表2所示。定义是最常见的，而关系约束虽然较少，但由于其固有的复杂性，匹配率较低。总体而言，定义的频率在23,000到34,000个实例之间，基本约束在15,000到31,000个实例之间，关系约束在7,000到19,000个实例之间。这些汇总结果表明，核心基础句子以高一致性生成，确保关键元素很少被遗漏。同时，关系约束的更多样化的匹配率通过引入多样性来丰富关卡，同时保持了一致的核心框架，这符合我们的预期。

表2：所有5,000个脚本中汇总的句子频率（不包括额外的基线测试）

句子类型	频率	平均匹配率（%）
定义	131000	82.36
基本约束	131000	57.77
关系约束	55000	25.38

总体而言，我们的结果表明大语言模型能够可靠地以高一致性复现设施定义，而复杂的关联约束则表现出更大的创造性和多样性。这种一致性验证了我们的方法，未来的改进可能涉及更高级的模型或一种结合自动化生成与针对性手动优化的混合方法。

4.2 优化测试

在从大语言模型输出中选定一个基准GLDL脚本后，我们将其集成到优化流程中，为每种优化方法生成10,000个最终布局配置。这些配置是通过应用先前引入的优化算法，在选定的参数和权重下生成的，确保每个生成的布局都受到定义的约束和环境条件的指导。

对于基于约束的优化，我们测试了模拟退火算法、遗传算法和贪婪方法，随后选择了性能最佳的方法进行后续任务。

4.2.1 参数和权重。我们采用了代表超市环境的维度设置（$W = 20m$，$L = 30m$，$H = 6m$），并将近/远阈值（5米和10米，分别）设置为确保适当的设施间距。对各种约束（例如重叠、边界违规和未对齐）分配了惩罚权重，以平衡它们的相对重要性。整体指标权重（w_p用于惩罚，w_c用于聚类程度，以及w_s用于稀疏性）被选择以在严格的约束遵守和空间质量之间保持平衡。附录D提供了关于惩罚计算的详细说明。

4.2.2 结果和分析。我们首先微调了惩罚和指标权重，以确保优化过程将提供理想的结果。在选择权重（如表4所示），这些权重通常产生稳定的数值结果和视觉上协调的布局后，我们通过为每种方法生成10,000个布局来评估三种优化方法——模拟退火、遗传算法和贪婪启发式——的性能，以确定我们基于约束的优化任务中最有效的方法。表3展示了每种方法的平均总加权指标值及其标准差。

表3：不同优化方法的加权总指标。值已相对于最大观测指标归一化到 [0,1] 尺度。缩写：SA = 模拟退火，GA = 遗传算法，贪婪 = 贪婪方法

方法	总指标	归一化指标	标准差
SA	35337.70	0.56	2912.642
贪婪	58947.89	0.94	13079.87
GA	62600.97	1.00	3948.396

SA 实现了最低的总加权指标（35,337.7），其次是贪婪算法（58,947.9），而 GA 得到了最高值（62,600.9）。虽然贪婪算法在计算速度上表现优于其他方法，但其较高的标准差（13,079 对比 SA 的 3,948）表明其解决方案的稳定性不如 SA。相比之下，SA 在数值性能和视觉质量之间保持了良好的平衡，使其成为此场景中最可靠的选择。因此，我们选择了 SA 进行进一步实验和分析。

聚焦于空间约束（SA），我们使用选定的惩罚和指标加权方案评估了10,000个生成的布局。如表4所示，邻近约束（惩罚_邻近和惩罚_远离）尽管权重适中（10和15），却贡献了31.70%和26.20%的总加权指标，突显了它们在塑造空间关系中的关键作用，同时在设施布局上提供了可控的灵活性。相比之下，分配给惩罚_重叠（30）和惩罚_边界（30）的高权重有效地确保了在环境边界内的无碰撞布局，这一点从它们微不足道的加权贡献（0.05%和0.003%）中得到了证明。

表 4：来自 10,000 个带加权平均的生成布局的惩罚权重和平均指标。

指标	权重	平均值	加权平均值
惩罚_xy	18	75.39	1357.02 (3.84%)
惩罚_z	20	49.06	981.20 (2.78%)
惩罚_邻近	10	1120.33	11203.30 (31.70%)
惩罚_远离	15	616.95	9254.25 (26.20%)
惩罚_可以_看见	2	4.79	9.58 (0.03%)
惩罚_重叠	30	0.59	17.70 (0.05%)
惩罚_边界	30	0.04	1.20 (0.003%)
惩罚_对齐	15	31.31	469.65 (1.33%)
惩罚_朝向	20	31.97	639.40 (1.81%)
惩罚_聚焦	10	9.42	94.20 (0.27%)
总_惩罚	1	24027.48	24027.48 (67.99%)
簇_度	100.0	77.57	7757.00 (21.95%)
稀疏度_指标	500.0	7.11	3555.00 (10.07%)
总_加权_指标			35337.70 (100%)

（30）和惩罚_边界（30）有效地确保了在环境边界内的无碰撞布局，这一点从它们微不足道的加权贡献（0.05%和0.003%）中得到了证明。与此同时，垂直放置约束（惩罚_z）和朝向要求贡献较小（2.78%和1.81%），反映了与理想配置的轻微但持续偏差。

聚类程度指标 ($w_c = 100.0$) 通过抑制过高的设施密度贡献了 21.95%，而稀疏度指标 ($w_s = 500.0$) 贡献了 10.07%，从而确保了空间分布的平衡。值得注意的是，所有惩罚项的总和占到了总加权指标的 67.99%，这验证了我们的约束驱动优化方法。这些结果表明，权重较低但违规频率较高的约束（如邻近规则）自然成为主要的成本驱动因素，而高权重惩罚（如重叠）则作为严格的护栏，由于其违规发生频率极低，对整体成本的影响最小。

惩罚权重与平均指标之间的成功对齐证实，我们的优化方法在生成游戏关卡布局方面是正确且有效的。优先考虑高权重约束确保了无碰撞、符合边界的布局，而平衡的聚类和稀疏度权重则促进了可导航的环境。未来工作可以探索基于布局复杂度或测试反馈的自适应权重策略，同时增强惩罚和空间指标以改善邻近符合度和空间连贯性。

4.3 Unity模拟测试

实验设置。 为了评估生成布局的可玩性和连贯性，我们将10,000个优化配置导入Unity环境以模拟实时游戏。一个AI智能体被设计用来模仿人类探索和交互，它会导航每个包含各种交互元素和敌人的关卡，目标是收集所有散落的钥匙、收集物品，并解锁出口以完成关卡。我们的AI

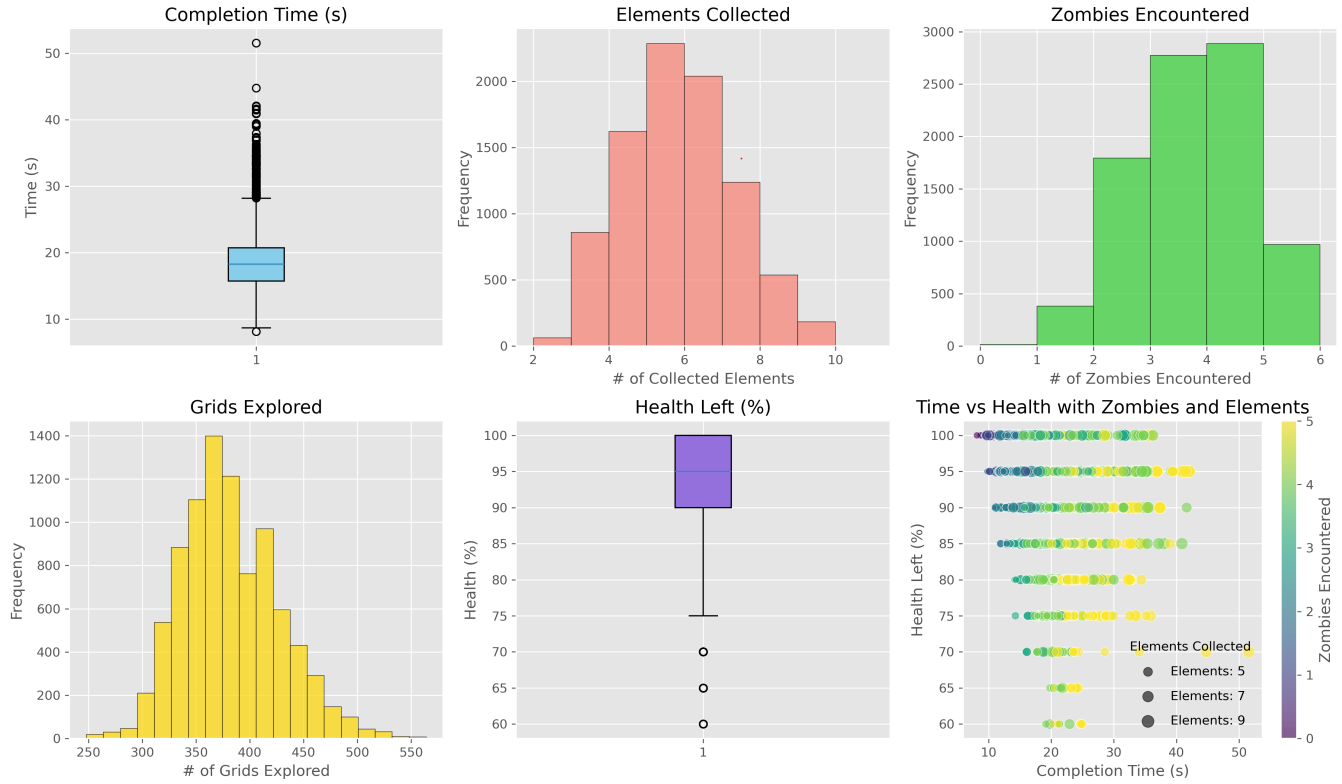


图2：关键模拟指标概述。完成时间（左上）和最终生命值（中间下）的箱线图反映了整体难度，而收集元素（中间上）、遭遇僵尸（右上）和探索网格（左下）的直方图则表明了关卡设计功能的用法和影响。右下角的散点图结合了完成时间、生命值和敌人遭遇，揭示了模拟环境中难度和内容利用如何相互交织。

逻辑（见算法1）如下：当智能体找到一个钥匙时，它会记录位置并优先寻找最近的钥匙；如果遇到敌人，智能体会停止战斗；如果物品在范围内，它们会被收集；如果没有可见的钥匙，智能体会移动到未探索网格单元最多的区域：

$$P^* = \arg\max_{P} \sum_{(i,j)\in \text{Grid}} \mathbb{1}_{\text{Unseen}(i,j)} \cdot e^{-\beta \cdot \text{dist}(P,(i,j))}$$

其中 $\mathbb{1}_{\text{Unseen}(i,j)}$ 如果网格单元未被看见则为1，否则为0，β 是一个优先考虑邻近单元的衰减因子，而 $\text{dist}(P,(i,j))$ 是 P 与网格单元 (i,j) 之间的欧几里得距离。

敌人被分配了不同的属性和参数，在进入其探测范围后会追逐和攻击智能体。如果智能体失去所有生命值、任何钥匙变得无法到达或某些路径无效阻碍关卡完成，则视为关卡失败。详细的参数配置在附录E中提供。

除了在10,000个主配置数据集上评估我们的框架外，我们还进行了两个子测试，每个子测试包含1,000次模拟，以展示可调性和可控性。在第一个子测试（战斗密度测试）中，大语言模型被提示创建一个敌人遭遇更频繁的场景。在第二个子测试（任务时长测试）中，大语言模型被指示生成关卡配置可能会延长代理的整体完成时间。两个子测试都遵循了之前描述的框架流程，导致相同的Unity模拟测试，具有相同的代理逻辑和环境参数。

结果与分析。 图2展示了10,000次主模拟的关键游戏指标。完成时间通常在10至25秒之间（占17%–42%），部分异常值超过50秒。收集元素的数量一般介于5至7个（平均5.38，占54%），僵尸遭遇平均为3.25（占54%）。大多数模拟探索的网格数在300至450个之间（占55%–82%），智能体健康值通常保持在90%以上。散点图进一步表明，完成时间越长与健康值越低、敌人遭遇越多相关。

表5总结了三种测试条件下平均指标，每个原始值均附带相应的归一化百分比。例如，在默认配置下，AI智能体平均用18.58秒（占31%）完成关卡，收集5.38个元素（占54%），遭遇3.25个僵尸（占54%），探索382.07个网格（占70%），并保留92.43%健康值。

为了可调性，生成了两个额外的GLDL脚本。战斗测试（增加敌人密度和复杂度）导致完成时间略有增加，僵尸遭遇显著上升（从54%到65%），最终生命值降至85.5%。相比之下，任务测试（将关键目标间隔得更远）将平均完成时间延长至25.13秒（42%）。

表5：Unity模拟测试结果

指标（最大值）	默认生成	战斗密度测试	任务持续时间测试
模拟次数	10,000	1,000	1,000
平均耗时（秒） (60)	18.58 (31.0%)	20.32 (33.9%)	25.13 (41.9%)
最小时间 (s) (60)	8.14 (13.6%)	10.04 (16.7%)	14.53 (24.2%)
最大时间 (s) (60)	51.55 (85.9%)	42.14 (70.2%)	55.42 (92.4%)
平均收集元素数 (10)	5.38 (53.8%)	5.47 (54.7%)	5.74 (57.4%)
平均遭遇僵尸 (6)	3.25 (54.2%)	3.89 (64.8%)	3.83 (63.8%)
平均探索网格 (550)	382.07 (69.5%)	408.37 (74.3%)	398.31 (72.4%)
平均剩余生命值（%） (100)	92.43 (92.4%)	85.54 (85.5%)	91.05 (91.1%)
无效级别			
超时	228 (2.28%)	12 (1.2%)	19 (1.9%)
不可达键	950 (9.5%)	78 (7.8%)	47 (4.7%)
环境详情			
宽度 (米)	20	20	20
长度 (米)	30	30	30
高度 (米)	6	6	6
交互元素	10	10	10
敌人	5	5	5
AI智能体健康值	100	100	100
时间阈值	60s	60s	60s

算法1AI智能体导航与交互逻辑

初始化：记忆 $\leftarrow \emptyset$
当游戏运行 **时**
  K $\leftarrow$检测按键()
  E $\leftarrow$检测敌人()
  I $\leftarrow$检测交互元素()
  记忆 $\leftarrow$ 记忆 $\cup$K
  如果 $\mathcal{E} \neq \emptyset$，**则**
    攻击敌人(E)
  否则如果 $\exists i \in \mathrm{I}$: 距离$(i) \leq$ 收集范围，**则**
    移动并收集(i)
  否则如果 记忆 $\neq \emptyset$，**则**
    $k^* =$ 最小化$_{k \in \text{记忆}}$ 距离(k)
    移动到k^*
    收集钥匙k^*
    记忆 $\leftarrow$ 记忆 $\setminus \{k^*\}$
    如果 所有钥匙已收集 $\wedge$ 出口可达 **则**
      移动到出口()
    结束如果
  else
    $P^* =$ 最大值$_P \sum_{(i,\ j) \in \text{网格}} \mathbb{1}_{\text{未见}(i,\ j)}\, e^{-\beta \cdot \text{距离}(P,\ (i,\ j))}$
    移动到(P^*)
  否则
结束循环

僵尸遭遇显著上升（从54%到65%），最终生命值降至85.5%。相比之下，任务测试（将关键目标间隔得更远），将平均完成时间延长至25.13秒（42%）。

并且略微提高了平均收集到的元素数量至5.74（57%）。这些结果表明，通过修改大语言模型提示或直接调整设施约束，该框架可以有效地进行调优以生成不同的游戏玩法配置文件。

此外，超过时间阈值的228个关卡和具有不可达键的950个关卡突出了未来优化的具体领域。总体而言，该系统达到了88.22%的有效率，表明大多数生成的关卡既是可玩的，也符合基本的设计约束。未来的工作将涉及人工测试以进一步验证这些自动评估。

5 结论

在本研究中，我们提出了一种集成大语言模型、专门的游戏关卡描述语言和设施布局优化的框架，以自动生成连贯、可玩的3D游戏关卡。从叙事级描述开始，该方法生成的布局反映了预期的空间约束和游戏玩法机制。基于代理的模拟证实，这些环境不仅可导航且引人入胜，而且保持了结构上的连贯性和可控性。

然而，当前框架存在局限性。虽然GLDL能有效编码基本约束，但它无法捕捉游戏设计师可能需要的复杂约束的完整范围——其中一些约束需要组合多个句子并进行额外的手动微调。类似地，假设房间和对象都可以表示为立方体的简化假设，过度简化了许多现实场景。潜在改进包括用最小的包含立方体近似不规则形状，使用预定义设施填充区域，或采用网格和高级几何表示，即使这些方法会增加计算开销。

此外，当前验证仅限于单一游戏类型（生存恐怖）和单一AI智能体玩法，可能无法完全捕捉其他游戏类型的多样化需求。

未来的工作包括将此框架扩展到更大规模的场景，结合时间维度，并引入更多游戏机制和逻辑元素以支持考虑游戏流程和叙事进展的动态场景。在多个游戏类型（例如，角色扮演游戏、策略游戏）中测试该框架，并结合特定类型的设施和约束，将进一步验证其泛化能力。采用具有不同行为特征（例如，基于潜行的导航与基于战斗的策略）的多样化AI智能体，将确保框架不会过度拟合单一玩法，并增强其对异构玩家交互的适应性。通过本地构建设施数据，该框架可以适应完全离线运行，减少对在线服务的依赖。通过解决这些挑战并利用本地资源，该方法可以演变为设计动态环境，以更好地满足当代游戏开发的复杂需求。

致谢

这项研究得到了NSERC探索基金RGPIN-2019-05213的支持。我们承认手稿的部分文本使用了生成式AI进行了修改，所有更改均由作者进行了审查。

参考文献

[1] Hanzhi Chen, Boyang Sun, Anran Liu, Jiaqi Gu, Kai Hormann, Stefan Leutenegger, 和 Marc Pollefeys. 2024. 城市规模的语义占用基准测试. 在 *2024 IEEE/RSJ 国际智能机器人与系统会议 (IROS)*. IEEE, 10264–10271. https://doi.org/10.1109/IROS58592.2024.10802505 [2] Ian Gumin, Do Heon Han, Stewart Morris, Seung Jean Yoo, Aditya Ganeshan, R. Kenny Jones, Qiuhong Anna Wei, Kailiang Fu, 和 Daniel Ritchie. 2024. 使用大语言模型程序合成和未整理的对象数据库进行开放世界室内场景生成. arXiv:2403.09675 [cs.CV] https: //arxiv.org/abs/ 2403 09675 . Abbas Ahmadi, Mir Saman Pishvaee, 和 Mohammad Reza Akbari Jokar. 2017. 多层设施布局问题的综述. 工业工程与计算机 107 2017 158 170 10 1016 2017 03 015 ( ), – . https://doi.org/ . /j.cie. . . anthropic. . 推出下一代Claude. https://www.anthropic.com/news/claude-3-family [4] Aren A. Babikian, Oszkár Semeráth, 和 Dániel Varró. 2024. 使用元启发式搜索将抽象交通场景规范具体化. *IEEE* 软件工程汇刊 50, 1 (2024), 48–68. https://doi.org/10.1109/TSE. 2023.3331254[5] Dave Epstein, Ben Poole, Ben Mildenhall, Alexei A. Efros, 和 Aleksander Holyn- ski. 2025. 基于布局学习的解耦3D场景生成. 在 第*41*届国际机器学习会议 *(ICML’24)*. JMLR.org, 奥地利维也纳, Article 500, 13页.[6] Chuan Fang, Yuan Dong, Kunming Luo, Xiaotao Hu, Rakesh Shrestha, 和 Ping Tan. 2024. Ctrl-Room: 基于布局约束的文本到3D房间网格生成. arXiv: 2310 03602 [ ] 2310 03602 . cs. CV https://arxiv.org/abs/ . Daniel J. Fremont, Tommaso Dreossi, Shromona Ghosh, Xiangyu Yue, Alberto L. Sangiovanni-Vincentelli, 和 Sanjit A. Seshia. 2019. Scenic: 一种场景规范和生成语言. 在 第*40*届*ACM SIGPLAN*会议 *on Programming Language Design and Implementation* (凤凰城, 亚利桑那州, 美国) *(PLDI 2019)*. 计算机协会, 美国纽约 , NY, USA, 63 78 10 1145 3314221 3314633 — . https://doi.org/ . / . Evan Hahn, Prosenjit Bose, 和 Anthony Whitehead. 2006. 持续实时建筑室内生成. 在 第*2006*届*ACM SIGGRAPH*视频游戏研讨会 (波士顿, 马萨诸塞州) *(Sandbox ’06)*. 计算机协会, 美国纽约, NY, USA, 179—-186. [9] Chengpeng Hu, Yunlong Zhao, 和 Jialin Liu. 2024. 基于大语言模型的关卡生成. 在 *2024 IEEE* 游戏 *(CoG)* 会议. 1–4. https://doi.org/ 10.1109/CoG60054.2024.10645597[10] Ziniu Hu, Ahmet Iscen, Aashi Jain, Thomas Kipf, Yisong Yue, David A Ross, Cordelia Schmid, 和 Alireza Fathi. 2024. SceneCraft: 一种用于将3D场景合成blender代码的大语言模型代理. 在 第*41*届国际机器学习会议 (奥地利维也纳) *(ICML’24)*. JMLR.org, Article 776, 31页.

[11] 蒋泽华, 艾尔·萨姆, 格林·迈克尔, 和托格尔·朱利安. 2022. 可控的3D关卡生成器学习. 在第 17 届数字游戏基础国际会议论文集（希腊雅典）（FDG ’22). 美国纽约 , NY, USA, ACM, 第71篇论文, 9页. [12] 哈利法·阿明, 邦特拉格·菲利普, 艾尔·萨姆, 和托格尔·朱利安. 2020. PCGRL: 基于强化学习的程序化内容生成. AAAI人工智能与交互式数字娱乐会议论文集 16, 1 2020 95 101 10 1609 16 1 7416 (10月), – . https://doi.org/ . /aiide.v i . 刘佳林 , 斯诺格拉夫 · 萨姆 , 哈利法 · 阿明 , 里西 · 塞巴斯蒂安, 约翰·乔治·N·亚纳卡基斯, 和托格尔·朱利安. 2021. 用于程序化内容生成的深度学习. 神经计算与应用 33, 1 (2021年1月), 19–37. https://doi.org/10.1007/s00521-020-05383-8[14] 洛佩斯·里卡多, 图滕内尔·蒂姆, 斯米尔克·鲁本·M., 德·克拉克·克拉斯·扬, 和比达拉·拉斐尔. 2010. 用于程序化楼层平面生成的约束增长方法. 在第11届智能游戏与模拟国际会议论文集 (GAMEON 2010). 英国莱斯特, 13–20. [15] 马丁·杰西. 2006. 程序化房屋生成：一种动态生成楼层平面方法. 在交互式计算机图形与游戏研讨会论文集. [16] OpenAI. 2024. 使用大语言模型进行推理学习. https://openai.com/index/learning-to-reason-with-llms/ [17] 普尔·瑞安和韦茨斯坦·戈登. 2024. 使用局部条件扩散进行组合式3D场景生成. 在2024年3D视觉国际会议 (3DV). 651–663. https://doi.org/10.1109/3DV62453.2024.00026[18] 任宣驰, 黄佳辉, 曾晓辉, 莫斯·肯, 菲德勒·桑贾, 和威廉姆斯·弗朗西斯. 2024. XCube: 使用稀疏体素层次结构的大规模3D生成建模. 在IEEE/CVF计算机视觉与模式识别会议论文集. 4209–4219. [19] 舒尔特·乔纳斯, 采·萨姆, 霍莱因·卢卡斯, 吴必晨, 王佳亮, 马·智瑶, 李坤鹏, 王晓方, 温巴沃·费利克斯, 何子健, 张佩昭, 莱比·巴斯蒂安, 瓦伊达·彼得, 和侯继. 2024. ControlRoom3D: 使用语义代理房间进行房间生成. 在IEEE计算机视觉与模式识别会议 (CVPR). 6201–6210. [20] 沙克 · 努尔 , 托格尔 · 朱利安 , 和尼尔森 · 马克 . 2016. 游戏中的程序化内容生成. 斯普林格. https://doi.org/ 10 1007 978 3 319 42716 4 . / - - - - S·P·辛格和R·R·K·夏尔马. . 设施布局问题不同方法的综述. 国际先进制造技术 30 (2006), 425–433. https: //doi.org/10.1007/s00170-005-0087-9[22] 苏姆米维尔·亚当, 斯诺格拉夫·萨姆, 古兹迪亚尔·马修, 霍尔姆加德·克里斯托弗, 海沃·艾米·K., 伊萨克森·阿伦, 尼尔·安德鲁, 和托格尔·朱利安. 2018. 基于机器学习的程序化内容生成 (PCGML). IEEE游戏 Transactions 10, 3 (2018), 257–270. https://doi.org/10.1109/TG.2018.2846639[23] 托德·格雷厄姆, 艾尔·萨姆, 纳西尔·穆罕默德·乌迈尔, 格林·迈克尔·C·, 和托格尔·朱利安. 2023. 基于大语言模型的关卡生成. 在第18届数字游戏基础国际会议论文集（葡萄牙里斯本）（FDG ’23). 美国纽约, NY, USA, ACM, 第 70, 8 10 1145 3582437 3587211 页. https://doi.org/ . / . . 托格尔·朱利安, 约翰·乔治·N·亚纳卡基斯, 斯坦利·肯·O., 和布朗·卡梅隆. 2011. 基于搜索的程序化内容生成：分类和综述. IEEE计算智能与游戏 Transactions 3, 3 (2011 172 186 10 1109 2011 2148116), – . https://doi.org/ . /TCIAIG. . 王一 , 罗杰良 , 高尔 · 亚当 , 阿瑟顿 · 伊万 , 和科赫 · 希尔马. 2024. PlotMap: 建筑游戏世界的自动布局设计. 在2024年IEEE游戏会议 (CoG). 1 8 10 1109 60054 2024 10645627 – . https://doi.org/ . /CoG . . 怀特黑德·吉姆. . 使用线性约束和SMT求解器进行程序化地牢的空间布局. 在第15届数字游戏基础国际会议论文集（马耳他布基巴）（FDG ’20). 美国纽约, NY, USA, ACM, 第101篇论文, 9页.[27] 吴振南, 李杨, 严韩, 商台章, 孙伟轩, 王森波, 崔瑞凯, 刘伟哲, 佐藤浩之, 李洪东, 和姬磐. 2024. BlockFusion: 使用潜在三平面外推进行可扩展的3D场景生成. ACM图形 Transactions 43, 4 (2024), 17页. https://doi.org/10.1145/3658188[28] 徐永志, 那洪·永洪, 王一夫, 萨·英奎, 杜云飞, 李杨, 姬磐 , 和李洪东 . 2024. Sketch2Scene: 从用户随意草图自动生成交互式 3D 游戏场景 . arXiv:2408.04567 [cs.CV] https: //arxiv.org/abs/ 2408 04567 . Yue Yang, 孙范云, 魏希斯·卢卡, 沃德·埃利, 赫拉斯蒂·阿尔瓦罗, 韩文森, 吴嘉俊, 哈伯·尼克, 克里希纳·拉纳吉, 刘凌杰, 卡利斯昂·克里斯, 亚茨卡·马克, 肯贝哈维·阿尼鲁达, 和克拉克·克里斯托弗. 2024. Holodeck: 语言指导的3D具身AI环境生成. 在IEEE/CVF计算机视觉与模式识别会议论文集. 16227–16237.[30] 赵伟欣·庄, 周坤, 李俊毅, 唐天毅, 王晓蕾, 侯宇鹏, 闵英倩, 张必晨, 张俊杰, 董子健, 杜奕帆, 杨晨, 陈宇硕, 陈志鹏, 蒋金昊, 任瑞阳, 李奕帆, 唐新宇, 刘子康, 刘培宇, 聂建云, 温继荣. 2024. 大语言模型综述. arXiv:2303.18223 [cs.CL] https://arxiv.org/abs/2303.18223

GLDL规范细节

本附录为维护文本中使用的游戏关卡描述语言（GLDL）的语法和格式提供了详细参考。

A.1 定义句子

定义句子用于声明和参数化构成游戏关卡的设施（对象或参考点）。其一般语法为：

$$[F = \text{定义}(\text{变量})]$$

F表示被定义的设施，而Definition关键字可以取三种形式：

A.1.1 预定义设施。

- 格式： $F =$ 预定义设施(P, O, N, T)
- 参数：– P (位置)：设施的固定位置– O (朝向)：设施面向的方向– N (数量)：实例数量– T (标签)：与设施关联的标签列表

A.1.2 新设施。

- 格式： $F = \text{NewFacility}(N, T)$
- 参数：– N (数量)：实例的数量– T (标签)：与设施关联的标签列表

A.1.3 NewPoint。

- 格式： $F = \text{NewPoint}(P)$
- 参数：– P (位置)：点的固定位置

A.2 约束语句

约束语句对游戏关卡内的设施施加空间或关系规则。其通用语法为：

$$[\text{约束}(F, \text{变量})]$$

其中 F 表示被约束的设施，而 Constraint 关键字可以有多种形式，例如：

A.2.1 Basic Constraints.

- **PlaceAroundPosition**：在指定位置邻近放置设施 P。格式：PlaceAroundPosition(F, P)
- **PlaceInRange**：将设施的放置限制在由位置 $P1$ 和 $P2$ 定义的边界框内。格式：PlaceInRange$(F, P1, P2)$
- **PlaceByWall**：要求设施必须放置在墙的邻近位置，并可选指定朝向 O。格式：PlaceByWall(F, O)

A.2.2 关系约束。

- **邻近**: 指定两个设施 $F1$ 和 $F2$ 应该彼此靠近。格式：邻近$(F1, F2)$
- **远离**: 指定两个设施 $F1$ 和 $F2$ 应该彼此远离。格式：远离$(F1, F2)$
- **可视**: 确保两个设施 $F1$ 和 $F2$ 之间存在清晰的视线。格式：CanSee$(F1, F2)$
- **聚焦**: 使一个设施 $F1$ 面向另一个设施 $F2$ 或位置 P。格式：Focus$(F1, F2)$ 或 Focus(F, P)
- **对齐**: 使两个设施 $F1$ 和 $F2$ 沿一个或多个轴对齐。格式：Alignment$(F1, F2)$ 或 Alignment$(F1, F2,$轴$)$
- **朝向**: 使两个设施 $F1$ 和 $F2$ 的朝向匹配。格式：Orientation$(F1, F2)$
- **侧置**: 将一个设施 $F1$ 放置在另一个设施 $F2$ 的旁边，相对于它们的朝向。格式：OnTheSide$(F1, F2)$
- **前置**：结合聚焦、朝向和对齐，将一个设施 $F1$ 放置在另一个设施 $F2$ 前面。格式：Front$(F1, F2)$
- **分组**：定义两个设施 $F1$ 和 $F2$ 之间的父子关系，具有相对位置 P 和朝向 O。格式：Group$(F1, F2, P, O)$

B 基于 LLM 的约束生成提示

本附录介绍了用于基于 LLM 的约束生成的结构化提示：

你是一位 3D 游戏关卡设计师，正在为设定在超市环境中的 3D 游戏创建详细关卡。使用游戏关卡描述语言 (GLDL) 来定义设施、指定它们的属性，并系统地应用约束。遵循这些指南以确保有效的 GLDL 输出：

1. GLDL 的目的：

- 控制设施的位置、朝向及其相互关系，以创建交互式环境。

2. 关键组件：

- **设施**：预定义设施、新设施、点。 • **变量**：使用 W、L、H 表示环境尺寸。
- **位置**：三元组 (x, y, z)；使用 None 表示无约束轴。

3. 定义句子：

- **预定义设施**：设施_名称 = 预定义设施(P=(x, y, z), O=朝向, N=数量, T=[标签],尺寸=(宽度, 长度, 高度))
- **新设施**：
  facility_name = NewFacility(

N=数字，T=[标签],维度=(宽度，长度，高度))

- **点**:
  点_名称 =新点(P=(x, y, z))

4. 约束句子:

- **放置**:
  –PlaceByWall(facility, 朝向)–
  PlaceInRange(facility, pos1, pos2)–
  PlaceAroundPosition(facility, 位置)
- **关系**:–邻近(facility1, facility2)–
  可视(facility1, facility2)–聚焦
  (facility, 目标)

5. 具体说明:

- 朝向值不要使用引号（例如，使用X而不是'X'）。
- 使用PlaceAroundPosition应用Z轴约束，对于无约束轴使用None。
- 将每个约束单独放在一行上。
- 确保所有引用的设施和点位都已预先定义。

6. 输出格式:

- 定义和约束每行一个。 • 排除W、L、H的定义。
- 严格遵循GLDL语法以实现自动解析。

根据上述指南生成一个包含所有设施定义和约束的GLDL文件，确保逻辑一致性并符合指定的游戏环境。

C Prompt for GameLevelDescription (后末日超市)

该关卡设定在一个受《最后生还者》启发的末日超市中。前置主入口是玩家进入的主要通道，而背面的紧急出口则可以作为可能的逃生路线。在入口附近，数个收银台沿墙排列，既充当障碍物，也提醒着商店昔日的日常运营。在商店中央，多排货架整齐排列，仍存放着部分商品，附近散落着购物车。一个显眼的商品展示区位于中心附近，突出展示特色商品，而墙上高处安装的监控摄像头则监视着主入口和收银台。

一个火灾警报器位于商店中心，一个垃圾桶被设置在收银台附近。沿着墙壁，一台自动售货机和一台饮水机提供小额补给。照明灯具分布在商店各处，一个小型洗手间紧靠入口处的一堵墙。弹药包、治疗包和文件出现在货架上，彼此间隔放置以鼓励搜索。现在有几只僵尸在这些过道里游荡，它们靠近重要物品或地点，对任何进入者构成严重威胁。

冷冻展示柜和冷藏展示柜位于中心附近，与货架排对齐，而库存袋则紧靠旁边。所有这些元素都保持在超市的墙内，以创造一种紧凑、具有挑战性的游戏体验。

D.惩罚计算细节

本附录详细解释了惩罚的推导方式。每种惩罚类型都是通过评估设施布局与特定约束条件来计算的。以下详细解释了每种惩罚的推导过程，并在适用情况下包含数学公式:

D.1 惩罚_x和惩罚_z

这些惩罚分别衡量沿X轴和Z轴偏离目标位置的程度。对于一个位于坐标(x, z)的设施，惩罚值计算为当前位置与目标位置之间的平方差:

$$\text{penalty_x} = w_x \cdot (x_{\text{current}} - x_{\text{target}})^2$$

$$\text{penalty_z} = w_z \cdot (z_{\text{current}} - z_{\text{target}})^2$$

where w_x和 w_z是分配给这些惩罚的权重。

D.2 惩罚_邻近和惩罚_远离

这些惩罚用于维护设施对之间指定的邻近阈值。对于每一对设施 (f_i, f_j)，计算欧几里得距离 d_{ij}:

$$d_{ij} = \sqrt{(x_i - x_j)^2 + (z_i - z_j)^2}$$

- **惩罚_邻近**: 当 $d_{ij} < d\text{min}$ 时应用，其中 $d\text{min}$ 是允许的最小距离。

$$\text{penalty_near+} = w_{\text{near}} \cdot (d_{\min} - d_{ij})^2 \quad \text{if } d_{ij} < d_{\min}$$

- **惩罚_远离**: 当 $d_{ij} > d\text{max}$ 时应用，其中 $d\text{max}$ 是允许的最大距离。

$$\text{penalty_far+} = w_{\text{far}} \cdot (d_{ij} - d_{\max})^2 \quad \text{if } d_{ij} > d_{\max}$$

where $w$near 和$w$far 是相应的权重。

D.3 惩罚_可以_看到

这个惩罚确保两个设施之间有清晰的视线。从一个设施的中心向另一个设施发射一条射线。如果射线与任何其他设施相交，惩罚会增加:

$$\text{penalty_can_see+} = w_{\text{can_see}} \cdot \mathbb{1}_{\text{obstructed}}$$

其中w可以_看到是权重，而 $\mathbb{1}$阻挡是一个指示函数，如果视线被阻挡则为1，否则为0。

D.4 惩罚_重叠

当设施在空间中相交时会产生重叠惩罚。计算两个设施之间的重叠 V重叠体积，惩罚与该体积成正比:

$$\text{penalty_overlap+} = w_{\text{overlap}} \cdot V_{\text{overlap}}$$

w重叠是分配的权重。

D.5 惩罚_边界

该惩罚确保所有设施都保持在定义的环境边界内。对于一个超出边界的设施，从其位置到最近的边界的平方距离 d边界会被计算：

$$\text{penalty_bounds+} = w_{\text{bounds}} \cdot d_{\text{bound}}^2 \quad \text{if out_of_bounds}$$

其中w边界是权重，而out_of_边界是一个指示边界违规的条件。

D.6 惩罚_对齐与惩罚_朝向

这些惩罚评估设施与指定轴对齐的程度或共享相同朝向的情况。对于沿轴的对齐，测量的偏差 θ 是期望对齐角度的偏差：

$$\text{penalty_alignment+} = w_{\text{alignment}} \cdot \theta^2$$

类似地，为了朝向一致性：

$$\text{penalty_orientation+} = w_{\text{orientation}} \cdot \theta^2_{\text{orientation}}$$

其中 w对齐 和w朝向 分别是相应的权重。

D.7 Penalty_聚焦

聚焦惩罚考虑设施朝向方向与目标方向之间的角度。如果该角度 ϕ 超过阈值ϕ阈值，惩罚将随偏差呈二次方增长：

$$\text{penalty_focus+} = \begin{cases} w_{\text{focus}} \cdot (\phi - \phi_{\text{threshold}})^2 & \text{if } \phi > \phi_{\text{threshold}} \\ 0 & \text{otherwise} \end{cases}$$

其中w聚焦 是权重。

D.8 Aggregating Penalties

最终目标函数将所有惩罚项聚合为加权求和：

$$\text{TotalMetric} = w_p \cdot P + w_c \cdot C + w_s \cdot S$$

其中：

- $P = \sum\text{PenaltyTerms}$表示总空间惩罚。• C 是聚类程度。• S 是稀疏度指标。
- w_p, w_c和 w_s分别是惩罚项、聚类程度和稀疏度指标的权重。

调整这些权重允许设计师优先考虑关卡布局的不同方面，确保优化过程与期望的设计目标一致。

这些惩罚计算共同塑造了最终设施布局。通过调整惩罚权重和约束，设计师可以将优化引导至优先考虑某些方面（例如，最小化重叠、确保聚焦或保持适当对齐）而非其他方面的布局，从而得到更定制化、更协调的3D游戏关卡环境。

E 模拟参数

本附录提供了在Unity模拟测试中使用的详细参数配置。这些参数按僵尸类型、NavMesh智能体默认设置和AI智能体设置进行分类，以确保清晰度和便于参考。

表 6：僵尸类型及其参数。缩写：BZ = 膨胀僵尸，CZ = 爬行僵尸，CKZ = 点击僵尸，GZ = 守卫僵尸，RZ = 奔跑僵尸

参数	BZ	CZ	CKZ	GZ	RZ
探测范围（米）	10	20	10	20	20
视场角（度）	360	120	360	180	120
攻击范围（米）	4	3	3	3	3
攻击伤害	20	10	15	10	5
攻击冷却（秒）	2	1	2	1	1
移动速度（米/秒）	2.5	4	3	4	6
射线投射扩散角（度）	30	30	30	30	30

表7：AI智能体和僵尸的NavMesh智能体默认设置

参数	值
速度（米/秒）	3.5
角速度（°/秒）	120
加速度（米/秒²）	8
制动距离（米）	0
障碍物避让半径（米）	0.5
障碍物规避高度 (米)	2

表8：AI智能体设置

参数	值
最大生命值	100
射击间隔 (秒)	1.5
僵尸探测范围（米）	3
物体收集范围（米）	4
抛射物速度（米/秒）	5
抛射物伤害	50

- **僵尸类型**：定义了不同僵尸变体的特征，包括它们的探测能力、攻击机制、移动速度和感官参数。
- **NavMesh智能体默认设置**：标准设置应用于所有NavMesh智能体，包括AI智能体和僵尸，控制它们的导航和避障行为。
- **AI智能体设置**：针对负责模拟人类式探索和游戏关卡内交互的AI智能体的特定配置，包括生命值、战斗能力和交互范围。

这些参数配置经过精心选择，以确保AI智能体表现出的行为与人类玩家一致，能够有效探索环境、与元素互动并应对敌人。僵尸类型的多样性带来了不同的挑战，要求AI智能体相应调整其策略。NavMesh设置促进了逼真的移动和障碍物规避，而AI智能体设置则在生存能力与达成目标的有效性之间取得了平衡。